



CONTROLLED PRODUCT BASELINE

Binding rules for product behavior, architecture, PostgreSQL data design, React and Fastify contracts, imports, reporting, administration, testing, releases, and five-year governance.

Document ID	INV-FWK-001
Status	DRAFT - Approval Required
Product Owner	Gaurav Mehta
Approval Authority	Gaurav Mehta
Issue Date	August 11, 2026
Implementation Candidate	Existing Saturday build; repository release metadata 1.3.2 at issue date
Classification	Internal
Validity	Five years from Product Owner approval unless amended

Release and application changes remain frozen until the approval page is signed.

1. Document Control

Governing status

This document is not effective until signed by Gaurav Mehta. Until then, the current build is an implementation candidate only and releases remain frozen.

1.1 Authority and review model

Gaurav Mehta is Product Owner, document owner, and approval authority. DBA and Operations feedback is incorporated as technical review evidence. Their review does not create a second signature requirement for v1.0.

Role	Named owner	Decision right	Required evidence
Product Owner	Gaurav Mehta	Approves, rejects, or amends framework and product scope	Signed approval and controlled change record
Engineering	Implementation team	Implements only approved requirements	Code, tests, conformance report
DBA review	DBA team evidence	Reviews normalization, keys, indexes, constraints, deletion, ownership	Catalog and migration review findings
Operations review	Operations team evidence	Reviews services, IIS, health, logs, update and support behavior	Runbook and operational acceptance evidence

1.2 Revision history

Version	Date	Status	Author/Owner	Summary
1.0	2026-08-11	Draft for approval	Gaurav Mehta	Initial binding product and engineering framework based on the Saturday implementation candidate and documented review evidence.

1.3 Amendment procedure

- A change request identifies the affected requirement IDs and the business reason.
- Impact analysis covers database, API, registry, permissions, imports, reports, activity, UI, tests, operations, documentation, and migration compatibility.
- Changes are implemented outside production, tested, and reconciled in a generated conformance report.
- The Product Owner approves the amendment before it changes the governing baseline.
- Failed acceptance returns the candidate to development; it does not modify this framework or consume a release version.

1.4 Requirement language

Term	Meaning
Must / shall	Binding requirement. Failure blocks acceptance.
May	Permitted but not required; still subject to security, compatibility, and change control.
Candidate evidence	Observed in source or scripts at issue date; not proof of production acceptance.
Acceptance evidence	Repeatable result produced against the candidate through the production IIS path.
Applicable	Requirement is in scope for the candidate. Non-applicability requires Product Owner approval and written rationale.

Contents

1. Document Control	2
1.1 Authority and review model	2
1.2 Revision history	2
1.3 Amendment procedure	2
1.4 Requirement language	2
Contents	3
2. Product Charter	5
2.1 Purpose	5
2.2 Users and outcomes	5
2.3 Scope	5
2.4 Success measures	5
2.5 Terminology	5
3. Architecture Standard	6
3.1 Responsibility boundaries	6
3.2 Ports and dependencies	6
3.3 Prohibited patterns	6
4. Domain Model	7
4.1 Cardinality and ownership	7
4.2 Asset and model boundary	7
4.3 Location hierarchy	7
5. Database Specification	8
5.1 Schema, ownership, and grants	8
5.2 Normalization standard	8
5.3 Table catalogue and delete behavior	8
5.4 Index standard	9
5.5 Constraint and deletion policy	9
6. Data Dictionary	10
6.1 Multiple reference contract	10
6.2 Lifecycle statuses	11
6.3 Data invariants	11
7. Profile Registry Contract	12
7.1 Required metadata	12
7.2 Administrative extension workflow	12
7.3 Propagation acceptance	12
8. Permissions Matrix	13
8.1 Phase 1 identities	13
8.2 Session and audit rules	13
9. Capability Specifications	14
9.1 Asset operation state rules	14

9.2 Import state machine 14

9.3 Guided import correction 14

9.4 Management attention dashboard 14

10. API Contract 15

10.1 Stable response rules 15

10.2 Compatibility 15

10.3 Request-to-route reconciliation 15

11. UI System 16

11.1 Locked visual foundations 16

11.2 Responsive acceptance 16

11.3 Page-state standard 16

11.4 Overlay and motion standard 16

11.5 Wording standard 16

12. Operations Standard 17

12.1 Runtime ownership 17

12.2 Supported operations 17

12.3 Observability and support 17

13. Acceptance Matrix 18

13.1 Mandatory scenario walkthroughs 22

13.2 Paperwork quality gate 22

13.3 Integrated go-live decision 23

14. Current Candidate Assessment 24

14.1 Evidence present 24

14.2 Release blockers and conformance backlog seeds 24

14.3 Approval consequence 24

15. Five-Year Roadmap 25

15.1 Roadmap governance 25

15.2 Backup plan if the candidate cannot conform 25

16. Architectural Reference Evidence 26

16.1 Internal candidate evidence 26

17. Approval 27

2. Product Charter

2.1 Purpose

Inventory Project controls laboratory and engineering inventory from receipt through availability, assignment, transfer, rework, archival, and disposal. It replaces spreadsheet-style row construction with asset-centered records, governed configuration, auditable operations, and profile-driven workflows.

2.2 Users and outcomes

Audience	Primary outcome	Daily decisions
User	Find trustworthy inventory and history; print labels	Where is an asset? What is its state? Who owns it?
Super User	Operate inventory and import/export governed data	Can this asset be assigned, returned, transferred, updated, or included in a batch?
Privileged Administrator	Safely evolve profiles, lookups, users, and configuration	Will this change remain compatible across every surface and existing record?
Management	Understand availability, risk, quality, and operational health	What requires attention, why, at what priority, and which records support the result?

2.3 Scope

Area	Controlled scope
In scope	Asset/model separation; categories and profiles; standard and dynamic fields; controlled lookups; hierarchical locations; users and roles; asset operations; relationships; imports; reports; activity; model images; labels and printing; IIS/Node/PostgreSQL operations.
Phase 1 identity	Database-backed company-network user picker. It provides application accountability but is not enterprise authentication.
Explicitly excluded	Requests workflow, generic sheets/rows/columns builder, browser database access, automatic runtime DDL, memory-mode production, user-created backups, SSO before Phase 2 approval, and ungoverned deletion.

2.4 Success measures

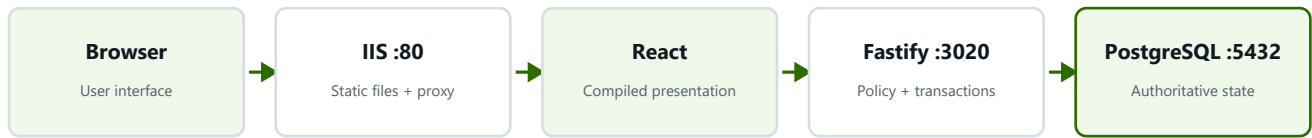
- Users locate an exact asset and understand its model, state, location, owner, references, and history without ambiguity.
- Administrators add a field or controlled value once and every enabled surface adopts it automatically.
- Imports are resumable, explain every issue, commit atomically, and refresh all affected views.
- Report KPIs reconcile exactly with drilldowns and exported rows.
- Every mutation is attributable, navigable, and recoverable through an immutable Activity record.
- A release is approved by evidence, not by an HTTP response or a version label.

2.5 Terminology

Term	Binding definition
Asset	One physical or logical inventory instance with unique serial, lifecycle state, owner, optional location, references, and revision.
Asset Model	Shared product identity and manufacturer/SKU/model metadata used by one or more assets.
Category	A named inventory family that selects one active profile.
Profile	Versioned fieldset controlling data entry, filters, details, imports, reports, and exports for a category.
Field	A governed semantic definition with stable key, type, validation, storage, aliases, and surface visibility.
Lookup	A controlled list and its active, aliased, ordered values.
Import Session	Persistent staged batch with pinned profile version, mappings, validation, corrections, and atomic commit state.
Activity	Immutable event plus structured changes linking actor and action to the affected record.
Archive	Reversible inactive asset state. It is not physical deletion.

3. Architecture Standard

PRODUCTION REQUEST PATH



No Vite, gateway, presentation server, memory database, runtime DDL, or browser-to-database access.

3.1 Responsibility boundaries

Component	Owns	Must not own
Browser / React	Presentation, interaction, local transient state, accessibility, route recovery	Permissions, validation truth, schema, statuses, report semantics, durable data
IIS	Public HTTP, static React files, /api/v1 reverse proxy, maintenance response	Business logic, database credentials, schema change
Fastify	Authorization, validation, transactions, concurrency, response normalization, activity, request IDs	Undocumented runtime DDL, browser presentation policy
PostgreSQL	Authoritative relational state, constraints, indexes, roles, immutable ledgers, migration history	UI copy, page layout, client-only defaults
Operations	Service lifecycle, configuration, migration, health, logs, maintenance, delta release, cleanup	Direct record correction or untracked production editing

3.2 Ports and dependencies

Service	Binding	Exposure	Dependency
IIS	0.0.0.0 / port 80	Company network	Compiled web + healthy Fastify proxy
Fastify API	127.0.0.1 / port 3020	Server internal	PostgreSQL + production configuration
PostgreSQL	127.0.0.1 / port 5432	Server internal	Windows service + data directory
HTTPS	Port 443 after certificate approval	Future controlled configuration	Certificate, binding, redirect, trusted origin review

3.3 Prohibited patterns

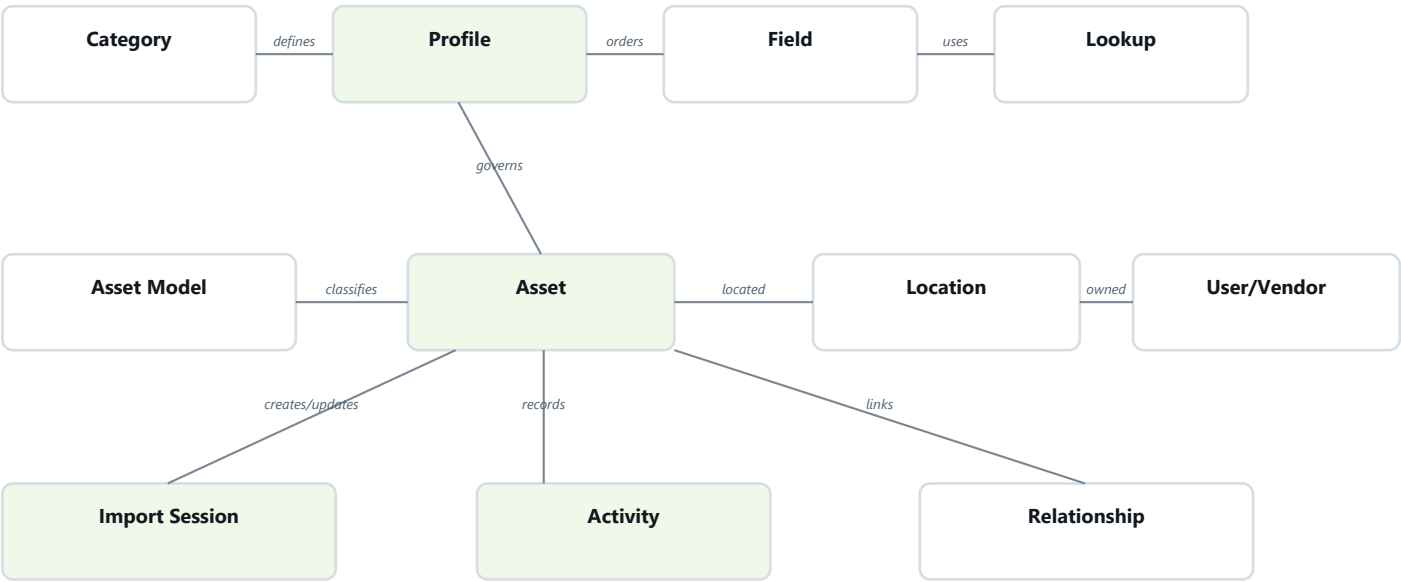
- No production Vite process, port 3021, port 8080 gateway, presentation server, or memory repository.
- No React hardcoded field contract, role, lifecycle status, controlled list, or report truth.
- No database change on API startup. Migration is an explicit, idempotent operations step.
- No direct browser connection to PostgreSQL and no PostgreSQL credential in compiled web assets.
- No page-specific replacement design system or accumulated responsive override layer.
- No compatibility claim unless web, API, schema, import contract, and package versions agree.

Architectural reference rule	Snipe-IT informs asset/model separation, fieldsets, permissions, imports, and activity. InvenTree informs persistent import sessions, stock-instance behavior, hierarchical locations, operations, and history. The Inventory Project remains a custom application and does not copy either product's code or UI.
------------------------------	---

4. Domain Model

AUTHORITATIVE DOMAIN RELATIONSHIPS

Identifiers form relationships; display labels never substitute for foreign keys.



4.1 Cardinality and ownership

Parent	Relationship	Child	Cardinality / rule
Category	selects	Asset Profile	One category has versions; at most one active profile.
Asset Profile	orders	Profile Field	One-to-many; field definition may be reused by many profiles.
Asset Model	classifies	Asset	One model to many instances; every asset has exactly one model.
Location	contains	Location	Recursive one-to-many; hierarchy type order is validated.
Asset	stores	Dynamic Field Value	Zero-to-many; value lifecycle is owned by asset.
Asset	references	External Reference	Zero-to-many per type; normalized uniqueness per asset/type/value.
Asset	participates in	Assignment/Status/Transfer	Zero-to-many immutable operational history.
Asset	links	Asset	Directed many-to-many relationship; parent cannot equal child.
Import Batch	contains	Rows / Issues / Results	One-to-many; session persists until completed or cancelled.
Activity Event	contains	Field Changes	One-to-many immutable structured changes.

4.2 Asset and model boundary

Asset Model (shared)	Asset (instance)
Category; manufacturer; model number; product name; board SKU; GPU SKU; architecture; specifications; image	Serial number; asset tag; status; owner; vendor; optional location; date received; milestone; project; notes; references; revision; archive state

4.3 Location hierarchy

Locations are stored as nodes with parent_id and location_type_id. The allowed path is Building -> Lab/Room -> Rack -> RU -> Cabinet/Storage. A location may stop at any valid level. Location is optional for an asset and for import; an unknown optional source location is a warning, not a blocking error.

5. Database Specification

Authoritative database

PostgreSQL database inventory_project is the only production data store. Application objects will move from public to dedicated schema invmgmt through one reviewed, data-preserving migration. The migration is a conformance backlog item after this framework is approved.

5.1 Schema, ownership, and grants

- Schema invmgmt is owned by the application owner role; runtime credentials receive only required DML and function execution privileges.
- PUBLIC has no CREATE privilege on invmgmt and no direct access to business tables.
- Migrations execute under a separate controlled migration identity, never through the browser or API startup.
- All functions, views, search_path settings, grants, sequences, indexes, and foreign-key references are reconciled during the public-to-invmgmt migration.
- The migration records one unique ID, is transactional where PostgreSQL permits, and is tested against a copy before production approval.

5.2 Normalization standard

The schema targets third normal form for core business records: identity, profile configuration, model metadata, assets, locations, operations, imports, reporting definitions, and activity are separate relations. Flexible field values do not replace normalized columns for high-value identifiers and relationships. Lookup labels remain display data; foreign keys store identity.

5.3 Table catalogue and delete behavior

The candidate production baseline defines 37 tables. The catalogue below is the binding review inventory; acceptance reconciles it against PostgreSQL system catalogs after migration.

Table	Primary key	Foreign key / delete rule	Purpose
schema_migrations	migration_key	None	Applied migration ledger; immutable IDs.
roles	id	None	Named application roles.
permissions	id	None	Atomic backend permissions.
role_permissions	role_id + permission_id	roles CASCADE; permissions CASCADE	Role-to-permission join.
application_users	id	None	Phase 1 user directory and active state.
application_user_roles	user_id + role_id	users CASCADE; roles RESTRICT	Auditable role assignments.
application_sessions	id (UUID)	users CASCADE	Server-side authenticated sessions and expiry.
lookup_lists	id	None	Controlled-list definitions and ownership.
lookup_values	id	lookup_lists RESTRICT	Values, aliases, ordering, active/deprecated state.
categories	id	None	Asset category registry.
asset_profiles	id	categories RESTRICT	Category fieldset/profile definition.
field_definitions	id	lookup_lists RESTRICT when controlled	Global field semantics, validation, visibility, and storage.
profile_fields	id	profiles CASCADE; fields RESTRICT	Ordered field use and required state per profile.
profile_versions	id	profiles RESTRICT; users RESTRICT	Immutable profile snapshots for imports and history.
manufacturers	id	None	Manufacturer registry; deactivate rather than delete.
vendors	id	None	Vendor registry; deactivate rather than delete.
asset_models	id	categories RESTRICT; manufacturers RESTRICT	Shared model/SKU metadata and image.
location_types	id	None	Ordered Building/Lab/Rack/RU/Cabinet type registry.
locations	id	parent locations RESTRICT; location_types RESTRICT	Hierarchical location nodes.
assets	id	model/profile/status/location/owner/vendor RESTRICT	Inventory instance, revision, archive state.

Table	Primary key	Foreign key / delete rule	Purpose
asset_field_values	id	assets CASCADE; fields RESTRICT	Dynamic instance values owned by an asset.
external_reference_types	id	None	NVBug, MRS, and Capacity reference definitions.
asset_external_references	id	assets CASCADE; type RESTRICT	Normalized, individual, timestamped references.
asset_assignments	id	asset/user/location/actor RESTRICT	Open and returned assignment history.
asset_status_events	id	asset/status/actor RESTRICT	Lifecycle status transition history.
asset_transfers	id	asset/owners/locations/actor RESTRICT	Owner and/or location transfer history.
asset_relationships	id	parent/child assets and actor RESTRICT	Installed-in, connected-to, or related asset links.
import_profiles	id	profiles RESTRICT	Import behavior tied to a profile.
import_batches	id (UUID)	import_profile/user RESTRICT	Persistent resumable import session.
import_column_mappings	id	batch CASCADE; field RESTRICT	Explicit source-column mapping.
import_batch_rows	id	batch CASCADE; target/committed asset RESTRICT	Staged source row, normalized values, inclusion, revision.
import_validation_issues	id	row CASCADE; resolver RESTRICT	Warning, blocking, and configuration issues and resolution.
import_commit_results	id	batch/row CASCADE; asset RESTRICT	Idempotent commit result per included row.
saved_reports	id	owner RESTRICT	Saved report filters, columns, sorting, and compatibility.
export_definitions	id	profile RESTRICT when scoped	Governed export layouts.
activity_events	id	actor/parent event/import RESTRICT	Immutable navigable mutation and access events.
activity_field_changes	id	activity event CASCADE	Structured before/after field changes.

5.4 Index standard

Query family	Required index behavior
Asset identity	Unique serial; partial unique populated asset tag; model lookup.
Operational filters	Status, location, owner, date received, active/archive state as justified by query plans.
References	Reference type + normalized value; asset + type + value uniqueness; newest-first retrieval support.
Locations	Parent + active + order; path/hierarchy query plan review.
Assignments	One open assignment per asset through a partial unique index.
Imports	Creator + status + updated time; batch + row order + inclusion; issue severity; unique mapping target.
Activity	Record type/id/time; actor/time; action/time; parent import where required.
Reporting	Index changes must be supported by EXPLAIN evidence and not duplicate existing coverage.

5.5 Constraint and deletion policy

- CHECK constraints enforce status/mode/severity enumerations, non-empty normalized identifiers, valid row status, non-self relationships, and revision bounds.
- UNIQUE constraints prevent duplicate serials, populated asset tags, role/permission joins, profile field order, lookup keys, external references, import mappings, and idempotent commit rows.
- Business records with dependencies are deactivated, deprecated, archived, or ended. Their APIs do not expose physical delete.
- An accidental DELETE attempted outside approved lifecycle rules is denied by permissions, foreign keys, or an explicit protective trigger where DBA review demonstrates residual risk.
- Activity events and changes reject UPDATE and DELETE through database enforcement in addition to API policy.

6. Data Dictionary

Contract count

The standard contract is exactly 19 fields: 8 required and 11 optional. Capacity Request # means Internal Capacity Request Number. Six lifecycle statuses are controlled separately.

Field key	Label	Requirement	Type	Storage	Definition and import rule
mrs_order	MRS order #	Optional	Multi-reference	external_reference:MRS_ORDER	Material Request System order reference. Canonical header and key accepted; configured aliases may extend mapping.
nbugs	NVBugs #	Required	Multi-reference	external_reference:NVBUG	Internal NVIDIA bug, enhancement, or task reference. Normalize, deduplicate, and preserve individually.
capacity_request	Capacity Request #	Optional	Multi-reference	external_reference:CAPACITY_REQUEST	Internal Capacity Request Number.
date_received	Date Received	Required	Date	assets.date_received	Day 0 receipt date. ISO date in API; locale-friendly display in UI.
board_sku	Board SKU	Optional	Text	asset_models.board_sku	Board or card assembly code shared by the asset model.
gpu_sku	GPU SKU	Optional	Text	asset_models.gpu_sku	GPU or chip variant code shared by the asset model.
model_number	Model #	Required	Text	asset_models.model_number	Manufacturer or internal model identifier used to resolve shared model metadata.
serial_number	Serial #	Required	Text / unique	assets.serial_number	Unique physical or logical instance identifier. Exact match key for Update Existing import mode.
milestone	Milestone	Optional	Text	assets.milestone	Engineering checkpoint or maturity state associated with the asset instance.
product_name	Product Name	Required	Text	asset_models.product_name	Human-readable product name shared by the asset model.
location	Location	Optional	Entity	assets.location_id	Foreign key to the hierarchical location tree. Blank never blocks import.
asset_status	Status	Required	Controlled value	assets.status_value_id	Lifecycle status from the approved six-value status list.
board_architecture	Board Architecture	Optional	Controlled value	asset_models.board_architecture	Approved architecture classification. Blank is valid; aliases are registry-managed.
pool_team	Pool/Team	Optional	Controlled value	assets.pool_team	Organizational pool or team responsible for operational use. Blank is valid.
project	Project	Optional	Text	assets.project	Project or operational activity associated with the asset.
asset_tag	Asset Tag #	Optional	Text / unique when populated	assets.asset_tag	Organization-issued asset identifier. Blank allowed; populated values unique.
owner	Owner / Assignee	Required	Entity	assets.owner_user_id	Foreign key to the selected application user responsible for the asset.
notes	Notes	Optional	Long text	assets.notes	Free text up to the configured limit. Never treated as a controlled value.
vendor	Vendor	Required	Entity	assets.vendor_id	Foreign key to the approved vendor record.

6.1 Multiple reference contract

- NVBug, MRS, and Capacity source input accepts comma, semicolon, or newline separators.
- Each value is trimmed, normalized, deduplicated, and stored as an individual relational row.
- Update preserves unchanged rows and their created timestamps, deletes only removed references, and inserts only new references.
- API arrays are ordered created_at descending then id descending. UI displays comma-separated newest-first values.
- Compact results show the newest reference and an additional-count indicator; asset detail shows complete grouped lists.
- Every reference mutation produces structured Activity before/after values.

6.2 Lifecycle statuses

Status	Availability group	Binding meaning
AVAILABLE	Available	On hand, usable, not assigned, blocked, archived, or pending corrective action.
GPU_READY	Available	Prepared, verified, and approved for GPU-related use, deployment, testing, or allocation.
IN_USE	Unavailable	Assigned, deployed, installed, or actively used; entered through Assign rather than a status-only shortcut.
REWORK	Unavailable / Exception	Requires repair, testing, cleaning, reconfiguration, relabeling, or corrective action.
E_WASTE	Unavailable / Exception	End of life and designated for electronic recycling, disposal, or removal.
ARCHIVE	Unavailable	Inactive but retained for history, compliance, audit, warranty, backup, or reference.

6.3 Data invariants

- Required state is enforced by the active profile and by database constraints where the value has a normalized storage column.
- An optional blank is represented as null or absence of a related/value row, not a placeholder dash or the text 'Empty value'.
- Controlled values store foreign keys or stable keys; display labels and aliases remain changeable presentation metadata.
- A model-level value is not copied into every asset instance. Instance overrides require an explicitly approved field and storage rule.
- Dates are exchanged as ISO 8601 values, stored in typed date/timestamp columns, and displayed in the user's approved locale format.
- Archived assets remain queryable by permission, retain references and history, and are excluded from active availability unless explicitly requested.

7. Profile Registry Contract

The registry is the application brain for data shape. React renders registry decisions; Fastify validates them; PostgreSQL stores definitions and versions. A hardcoded page-specific field list is a framework violation.

7.1 Required metadata

Property	Rule	Administrator caution
field_key	Stable machine identifier; immutable after first use	Changing a key breaks storage, imports, reports, and historical values; use a new field and deprecate the old one.
label / definition / help	Human meaning and concise recovery guidance	A label change is audited; definition cannot silently change semantic meaning.
field_type	Text, long text, number, date, boolean, controlled value, entity, multi-reference	Type changes require compatibility analysis and may be blocked when values exist.
required	Profile-specific required state	Making an existing field required requires completeness analysis and remediation before activation.
validation	Length, pattern, range, uniqueness, normalization	Server is authoritative; client mirrors only for immediate feedback.
storage mapping	Normalized column, foreign key, dynamic value, or external reference	Mapping must match type and query/report needs.
import aliases	Case-insensitive approved source headings	Ambiguous aliases are prohibited; collisions block activation.
surface flags	Add, Update, Filter, Detail, Import, Report, Export	An enabled surface must consume the field automatically.
active / deprecated	New use permitted or historical-only	Deactivation never destroys existing values.
profile version	Immutable snapshot of active behavior	Existing import sessions remain pinned and become stale when required.

7.2 Administrative extension workflow

Step	Binding action
1	Define field semantics, immutable key, type, storage, validation, aliases, and visibility.
2	Run dependency and completeness analysis against active profiles, stored values, imports, reports, and exports.
3	Add the field to a draft profile version and select required state and order.
4	Resolve controlled-list ownership and values when applicable.
5	Generate preview surfaces and run the automatic propagation suite.
6	Approve and activate the new profile version with reason and Activity event.
7	Mark unfinished import sessions Needs Revalidation when their pinned contract is affected.

7.3 Propagation acceptance

Twentieth-field test

A Privileged Administrator adds one optional test field to a draft profile. After activation, Add, Update, Filter, Detail, generated Import template, Import validation, Report field selection, and Export all expose it without a code change. The test field is then deprecated through the same workflow.

8. Permissions Matrix

UI visibility follows the server-derived permission map, but the Fastify pre-handler is the enforcement boundary. Direct API calls must produce the same authorization result as the UI.

Capability	User	Super User	Privileged Administrator	Permission
Search and view assets	Yes	Yes	Yes	asset.view
View asset history	Yes	Yes	Yes	asset.history
Print labels	Yes	Yes	Yes	label.print
Add asset	No	Yes	Yes	asset.create
Update asset	No	Yes	Yes	asset.update
Assign / Return / Transfer / Status	No	Yes	Yes	asset.operate
Manage asset relationships	No	Yes	Yes	asset.relationship
Manage model images	No	Yes	Yes	model.image
Execute import	No	Yes	Yes	import.execute
Approve permitted lookup from import	No	Yes	Yes	import.lookup.resolve
View reports	No	Yes	Yes	report.view
Export reports	No	Yes	Yes	report.export
View global activity	No	Yes	Yes	activity.view
Manage profiles and fields	No	No	Yes	admin.profile
Manage lookup lists and values	No	No	Yes	admin.lookup
Manage locations	No	No	Yes	admin.location
Manage users, roles, permissions	No	No	Yes	admin.identity
Manage system configuration	No	No	Yes	admin.system

8.1 Phase 1 identities

The user picker lists only active application users from PostgreSQL. All listed users receive User. Igor Margulis, Monica Martin, and Gaurav Mehta additionally receive Super User and Privileged Administrator. Guest is not a privileged identity. Phase 1 accountability is appropriate only for the company network; Entra ID SSO requires a later framework amendment.

8.2 Session and audit rules

- Login selects an active database user and creates a server-side session with expiry and CSRF controls.
- Mutation payloads cannot contain authoritative actor, role, or permission values.
- Fastify resolves effective permissions per request and records the selected user and effective authorization context.
- Role changes invalidate or refresh effective permission state according to the session contract.
- Authentication failure returns 401; authenticated but unauthorized operation returns 403; both use the stable error envelope.

9. Capability Specifications

Capability	Required production behavior
Search	Exact identifiers, free text, independent category shortcuts, hierarchical location and profile-driven filters, bulk selection, export, labels, and Add Asset when permitted.
Inventory	Paginated category browsing, lifecycle state, model/asset distinction, references, operations, relationships, refresh, and route-safe failures.
Import	Persistent create/update session, template, mapping, validation, guided correction, revalidation, atomic commit, resume, validation export, and automatic refresh.
Reports	Server totals, lifecycle, dimensions, quality, queues, aging, trends, charts, drilldowns, pagination, save, and CSV parity.
Activity	Actor, action, source, reason, reference, parent operation, structured changes, filters, pagination, and direct record links.
Admin	Profiles, fields, semantics, lookups, locations, identities, statuses, vendors, manufacturers, import profiles, model images, and configuration health.
Asset Workspace	Overview, update, Assign, Return, Transfer, Change Status, Archive, Restore, relationships, label, history, and conflict recovery.
Labels and Printing	Code 128 SVG from asset tag, physical preview, one label page per asset, browser printer/PDF support, print-only CSS, and activity.

9.1 Asset operation state rules

Action	Precondition	Atomic result
Assign	Asset is assignable; assignee valid; revision current	Owner/assignment/location/status revision update + Activity
Return	Open assignment exists; revision current	Close assignment; clear or replace location/owner per operation; status update + Activity
Transfer	At least owner or location changes; targets active; revision current	Transfer row + asset update + Activity
Change Status	Transition allowed; IN_USE requires Assign	Status event + asset revision + Activity
Archive	Permission and dependency rules satisfied	Archived timestamp/state + Activity; record retained
Restore	Archived asset remains valid under active profile	Active state restored with validated status + Activity

9.2 Import state machine

Allowed session states are Draft, Mapping, Validating, Needs Attention, Ready, Committing, Completed, Failed, Cancelled, and Needs Revalidation. The server owns transitions. A profile or lookup change marks unfinished affected sessions for revalidation; opening a session revalidates stale issues before display.

9.3 Guided import correction

- Non-empty unrecognized controlled values show the CSV value, closest approved values, Use Approved Value, Add As New Value when authorized, Correct In Source CSV, and direct row/field context.
- Add As New Value requires a reason, creates Activity, and revalidates every matching staged row.
- Missing controlled-list mappings show Repair Profile and link to the exact Admin field.
- The same rejected value may be bulk-corrected across rows. Users may edit, exclude, or restore staged rows.
- Validation CSV includes row, field, source value, severity, issue, and resolution.

9.4 Management attention dashboard

Super Users and Privileged Administrators receive an operational attention view sourced from server queries: failed or stale imports, rework backlog, e-waste queue, missing metadata, inactive configuration dependencies, recent failures, unassigned assets, service degradation, and report data freshness. Each item has severity, age, owner, count, and a direct record or filtered-result link.

10. API Contract

Domain	Endpoint family under /api/v1	Contract rule
Identity	GET /auth/users; POST /auth/login; GET /auth/session; POST /auth/logout; GET /session	Public picker only for users; session governs all protected calls.
Health and compatibility	GET /health/live; GET /health/ready; GET /version	Liveness, database readiness, and component contract.
Registry	GET /categories; GET /profiles/id; GET /lookups	Read-only profile-driven surfaces.
Assets	GET/POST /assets; GET/PATCH /assets/id; POST /assets/id/operations	Revision required for update; structured 409 on conflict.
Relationships and images	POST/DELETE /assets/id/relationships; PUT/DELETE /assets/id/model-image	Permission checked and audited.
Imports	POST/GET /imports; file; mappings; validate; row correction; bulk correction; lookups; commit; cancel; validation.csv	One import session envelope across the lifecycle.
Import templates	GET /profiles/id/import-template.csv	Generated from current active profile/version.
Reports	GET /reports; GET /reports/id/results; GET /reports/id/export	Shared query contract for KPI, drilldown, and export.
Activity	GET /activity; GET /assets/id/activity	Paginated and navigable with changes array.
Labels	GET /labels/:assetId; POST /labels/print	Code 128 SVG and print activity.
Admin	users, roles, profiles, fields, lookups, locations, statuses, vendors, manufacturers, images, health	Privileged Administrator only by backend permission.

10.1 Stable response rules

Contract	Required initialized structure
Asset	references.nvbugs[], references.mrsOrders[], references.capacityRequests[], fieldValues[], relationships[], revision
Activity	events[]; each event changes[] even when empty; pagination object
Report	kpis object, dimensions[], trends[], rows[], page, limit, total, filter echo, data timestamp
Import	session object, mappings[], rows[], issues[], summary, permissions, version state
Registry	categories[], profiles[], fields[], lookupLists[], values[]; active/version metadata
Error	status, code, message, details object/array, requestId; latest record for 409 where relevant

10.2 Compatibility

GET /api/v1/version returns packageVersion, webVersion, apiVersion, schemaVersion, importContractVersion, and compatible. The updater and frontend reject a mismatched contract before presenting business pages. The acceptance gate compares direct API and IIS-proxied responses and verifies the compiled web version.

10.3 Request-to-route reconciliation

A generated static check extracts frontend /api/v1 requests and backend route registrations. Every frontend call must map to one documented endpoint and every public endpoint must have ownership, permission, request schema, response schema, error cases, pagination rule, and test coverage.

11. UI System

11.1 Locked visual foundations

Foundation	Binding rule
Typography	One font family; stable rem-based scale; hero scale only for true hero; compact headings in tools and panels; no viewport-scaled fonts.
Spacing	4 px base rhythm with documented 4/8/12/16/24/32 values; page and component spacing use tokens only.
Color	Neutral working surface, NVIDIA green for clear emphasis, semantic status colors with text/icon redundancy; no one-note gradient palette.
Controls	Consistent 40-44 px default controls, labels above controls, inline validation, familiar icons with tooltips, accessible focus.
Cards	Repeated records and genuine framed tools only; radius 8 px or less; no card nesting or floating page sections.
Data density	Inventory and operational pages prioritize scanning, comparison, and repeated action over marketing composition.
Responsive	Stable max-width shell and grid constraints; components reflow, never shrink into illegibility; horizontal page scrolling prohibited.

11.2 Responsive acceptance

Viewport	Expected behavior
1920x1080	Centered bounded content; efficient rows; right-side workspace may be used without page shift.
1366x768 / 1280x720	Single content grid as needed; workspace and filters become bounded overlay sheets; actions remain visible.
768x1024	Tablet navigation and full-height sheet; one internal scroll surface; no clipped footer actions.
390x844	Single-column content; full-screen overlays; compact action menu; text wraps without overlap.

11.3 Page-state standard

Every route and major region implements Loading, Empty, Permission Denied, Configuration Required, Recoverable Failure, Conflict, and Success feedback where applicable. Skeletons preserve final dimensions. Route error boundaries isolate failure and show Retry, Return to Search, request ID, and support guidance. A malformed record is skipped or isolated with an explicit warning; it cannot blank the route.

11.4 Overlay and motion standard

- One overlay component serves filters, asset detail, add/update, labels, import corrections, and administrative editing.
- Backdrop click, Escape, close control, and Browser Back close the overlay; focus returns to the initiating control.
- Header and action footer remain visible; exactly one content region scrolls; background scroll is locked.
- Transitions are 120-220 ms, use opacity/transform where possible, never delay required work, and disable under prefers-reduced-motion.
- Status changes, import progress, saves, and errors use restrained visible feedback; decoration does not compete with data.

11.5 Wording standard

Use	Avoid	Reason
Asset	Inventory row	Represents a domain object, not a spreadsheet construct.
Assign / Return / Transfer / Change Status	Check In / Out or Workflow	Names the user's intent and resulting operation.
Advanced Filters	Intelligent filter	Describes a concrete action; fields remain profile-specific.
Needs Attention: follow the linked correction	Unexpected server error	Names the condition and provides a direct recovery path.
Configuration required: repair Board Architecture mapping	No approved values configured	Names responsibility and direct recovery step.
No assets match these filters	No data	Explains the current state and offers a clear action.

12. Operations Standard

CONTROLLED CHANGE AND RELEASE FLOW



A version number is assigned only after acceptance. A failed gate returns the candidate to development without changing this framework.

12.1 Runtime ownership

Layer	Owner artifact	Health signal	Failure action
IIS	Inventory Project website and rewrite/proxy configuration	Static index + proxied readiness	Maintenance response, inspect IIS logs and binding
Fastify	InventoryProjectApi Windows service and bundled Node runtime	live, ready, version, request/error rates	Restart service only after log/DB diagnosis
PostgreSQL	postgresql-x64-18 and inventory_project/invmgmt	Service, connection, transaction probe	Database escalation; do not reset or modify data manually
Release	Baseline + component delta + stable updater + release ledger	Hash, compatibility, migration and acceptance evidence	Reject/restore staged component; no direct live patch

12.2 Supported operations

Operation	Required behavior
Status	Shows service state, ports, version contract, database reachability, IIS proxy, current release and last acceptance.
Logs	Presents structured recent failures by request ID, record link, component, severity, and time; supports direct record navigation.
Maintenance	Activates user-safe maintenance response before a component switch and records operator/reason.
Migrate	Lists pending IDs, validates prerequisites, applies unique transactional migrations, records result, never starts from API.
Update	Verifies from/to version, updater version, component hashes, migrations, stages complete component, switches only declared components, checks health and acceptance.
Cleanup	Removes only successful package/extraction/temp directories; preserves application, DB, uploads, config, logs, and release ledger.
Failure	Restores held component when publishing fails, removes maintenance only after health, and leaves a clear failed ledger entry.

12.3 Observability and support

- Structured logs include timestamp, level, component, requestId, userId, action, recordType, recordId, route, status, duration, and safe error code.
- The UI Activity ledger is not a substitute for service logs; service logs are not a substitute for immutable business Activity.
- External monitoring checks public availability, API readiness, database reachability, p95 latency, error rate, and sustained resource pressure.
- Alerts define severity, owner, acknowledgment, escalation, and a direct diagnostics link.
- Support documentation includes common symptoms, exact evidence collection, safe actions, stop conditions, and escalation contacts/roles.

13. Acceptance Matrix

Every applicable requirement below must appear as Passed in a generated conformance report before packaging. 'Candidate evidence' is not accepted as a substitute for the listed result. The report records test name, build commit, environment, time, evidence location, and operator.

Requirement	Binding rule	Required acceptance evidence	Gate
GOV-001	This PDF is the governing product and engineering baseline after Product Owner approval.	Signed approval page and controlled document hash.	Product Owner
GOV-002	Gaurav Mehta is Product Owner and sole approval authority for v1.0.	Approval signature and date.	Product Owner
GOV-003	DBA and Operations feedback is retained as technical review evidence without separate approval authority.	Review evidence listed in the change record.	Product Owner
GOV-004	No release or application change proceeds while this framework is unapproved.	Release ledger contains no post-freeze production package.	Product Owner
GOV-005	Every change starts with a requirement ID or approved change request.	Change request links to affected requirement IDs.	Product Owner
GOV-006	Direct production edits, minified JavaScript patches, manual database corrections, and reused migration IDs are prohibited.	Repository and deployment evidence show packaged, unique, reviewed changes.	Product Owner
GOV-007	The approved framework remains effective for five years unless amended through change control.	Document control records approval and amendment history.	Product Owner
GOV-008	Source is committed and tagged before any implementation candidate is releasable.	Git commit, clean status, signed or protected tag.	Product Owner
ARCH-001	Production request path is Browser -> IIS -> compiled React -> Fastify /api/v1 -> PostgreSQL.	Architecture and runtime topology checks.	Automated + reviewed
ARCH-002	IIS owns public port 80; Fastify binds 127.0.0.1:3020; PostgreSQL binds 127.0.0.1:5432.	Port and service inspection after reboot.	Automated + reviewed
ARCH-003	Vite, gateway, presentation mode, memory persistence, direct browser database access, and runtime schema creation are prohibited in production.	Process, package, and configuration inventory.	Automated + reviewed
ARCH-004	React presents server state and never defines permissions, validation, statuses, fields, or report semantics.	Source conformance tests and API-driven rendering tests.	Automated + reviewed
ARCH-005	Fastify owns authorization, validation, transactions, concurrency, and stable response contracts.	Endpoint tests under every role and failure mode.	Automated + reviewed
ARCH-006	PostgreSQL is authoritative for inventory, configuration, permissions, imports, reporting definitions, and activity.	Persistence and restart tests.	Automated + reviewed
ARCH-007	Snipe-IT and InvenTree are architectural references, not copied code or user interface.	Design review traceability.	Automated + reviewed
ARCH-008	Dependencies and responsibility boundaries are recorded and owned.	Operations dependency inventory and owner matrix.	Automated + reviewed
ARCH-009	API and web compatibility are exposed by a version contract and checked before publication.	Version endpoint and updater compatibility check.	Automated + reviewed
ARCH-010	One locked component system governs all product pages and cosmetic changes.	Visual regression suite and token audit.	Automated + reviewed
DATA-001	Application objects reside in dedicated PostgreSQL schema invmgmt after one controlled migration from public.	Schema inventory, object ownership, grants, and migration evidence.	Automated + reviewed
DATA-002	Database inventory_project and schema invmgmt are distinct concepts; PostgreSQL remains the only database engine.	Connection configuration and catalog evidence.	Automated + reviewed
DATA-003	Every relational business record has a primary key and every relationship uses a foreign key.	Catalog query finds no missing required key.	Automated + reviewed
DATA-004	Display labels are not stored as relational identifiers.	Schema and query review.	Automated + reviewed
DATA-005	Physical deletion of referenced business records is prohibited; assets archive and configuration deactivates or deprecates.	Delete behavior and API tests.	Automated + reviewed
DATA-006	Business foreign keys use RESTRICT unless an approved lifecycle rule explicitly permits another behavior.	Foreign-key catalogue reconciliation.	Automated + reviewed
DATA-007	Technical children use CASCADE only when their parent is the sole owner of their lifecycle.	Foreign-key catalogue reconciliation.	Automated + reviewed
DATA-008	Serial number is required and globally unique; populated asset tag is unique.	Constraint and duplicate-write tests.	Automated + reviewed
DATA-009	Exactly 19 standard fields exist: eight required and eleven optional.	Registry count assertion and generated surface tests.	Automated + reviewed
DATA-010	Lifecycle statuses are exactly IN_USE, REWORK, E_WASTE, ARCHIVE, GPU_READY, and AVAILABLE until changed through governance.	Controlled-list count and value assertion.	Automated + reviewed

Requirement	Binding rule	Required acceptance evidence	Gate
DATA-011	Locations follow Building -> Lab/Room -> Rack -> RU -> Cabinet/Storage with nullable optional assignment.	Hierarchy constraints and traversal tests.	Automated + reviewed
DATA-012	Indexes support identifiers, status, model, location, owner, date, references, activity, and resumable imports.	Index catalogue and query-plan review.	Automated + reviewed
DATA-013	All business mutations are atomic, revision-controlled, permission-checked, and recorded in Activity.	Rollback, conflict, authorization, and activity tests.	Automated + reviewed
DATA-014	Migrations are immutable, unique, transactional, forward-only, and executed before application startup.	Migration ledger and failure rollback tests.	Automated + reviewed
DATA-015	Database ownership and grants follow least privilege; PUBLIC cannot create application objects in invmgmt.	Role, schema owner, and privilege catalogue.	Automated + reviewed
PROF-001	The profile registry is the only source of field behavior.	Cross-surface registry propagation test.	Automated + reviewed
PROF-002	A field definition includes key, label, definition, help text, type, validation, uniqueness, aliases, visibility, ordering, active state, and deprecation state.	Admin and database contract tests.	Automated + reviewed
PROF-003	Profile versions pin import sessions and preserve historical interpretation.	Version and stale-session tests.	Automated + reviewed
PROF-004	Administrator field additions propagate to Add, Update, Filter, Detail, Import, Report, and Export without code changes.	Twentieth-field end-to-end test.	Automated + reviewed
PROF-005	Field keys are immutable after first use; labels may change with audit.	Admin mutation tests.	Automated + reviewed
PROF-006	Invalid destructive profile changes with active dependencies are blocked clearly.	Dependency and deactivation tests.	Automated + reviewed
PROF-007	Dropdown values have aliases, active state, ordering, usage count, and ownership.	Admin lookup registry and import matching tests.	Automated + reviewed
PROF-008	Optional blank controlled fields never produce validation errors.	Create, update, filter, and import tests.	Automated + reviewed
PROF-009	Storage mappings are explicit and checked for compatibility with field type.	Registry health query and admin validation.	Automated + reviewed
PROF-010	Every enabled surface declares how it consumes each enabled field.	Conformance report surface matrix.	Automated + reviewed
AUTH-001	Phase 1 uses a database-backed company-network user picker without passwords or SSO.	Authentication mode and session tests.	Automated + reviewed
AUTH-002	Every listed member receives User; Igor Margulis, Monica Martin, and Gaurav Mehta also receive Super User and Privileged Administrator.	Seed and effective-role query.	Automated + reviewed
AUTH-003	The backend resolves actor, roles, and permissions from the session; clients cannot submit or spoof them.	Payload tampering and endpoint tests.	Automated + reviewed
AUTH-004	User may view assets, history, and print labels.	Role matrix endpoint suite.	Automated + reviewed
AUTH-005	Super User adds and updates assets, operates assets, imports, exports, views reports and activity, and resolves permitted lookup issues.	Role matrix endpoint suite.	Automated + reviewed
AUTH-006	Privileged Administrator manages profiles, lookups, locations, identities, roles, and system configuration.	Role matrix endpoint suite.	Automated + reviewed
AUTH-007	Unauthorized operations return a structured 403 and are not merely hidden in React.	Direct API authorization tests.	Automated + reviewed
AUTH-008	SSO is a later controlled phase and cannot be partially enabled in Phase 1.	Configuration and roadmap gate.	Automated + reviewed
ASSET-001	Asset Model stores shared manufacturer, SKU, model, specifications, and image; Asset stores instance identity and state.	Schema and API contract tests.	Automated + reviewed
ASSET-002	Create and Update are profile-driven and validate server-side.	Cross-surface and API tests.	Automated + reviewed
ASSET-003	User actions are Assign, Return, Transfer, Change Status, Archive, and Restore.	UI terminology and operation endpoint tests.	Automated + reviewed
ASSET-004	Assign, Return, Transfer, status, archive, and restore update state and Activity in one transaction.	Atomic rollback and event tests.	Automated + reviewed
ASSET-005	Asset updates require revision and stale edits return structured 409 Conflict.	Five-session concurrency test.	Automated + reviewed
ASSET-006	Relationships use explicit parent, child, and relationship type and prevent self-reference.	Constraint and API tests.	Automated + reviewed
ASSET-007	Model images are type, size, and dimension validated and have category fallback icons.	Upload and rendering tests.	Automated + reviewed

Requirement	Binding rule	Required acceptance evidence	Gate
ASSET-008	NVBug, MRS, and Capacity references are relational, normalized, deduplicated, and preserved individually.	Reference mutation and database tests.	Automated + reviewed
ASSET-009	Multiple references accept comma, semicolon, or newline separators and return newest first.	Create/update/detail/search ordering tests.	Automated + reviewed
ASSET-010	Compact cards show the newest reference plus additional count; details show all reference groups.	Page rendering tests.	Automated + reviewed
IMPORT-001	Imports use persistent, resumable server-side sessions.	Leave, resume, restart, and ownership tests.	Automated + reviewed
IMPORT-002	Create Assets and Update Existing are separate modes; update matches exact unique Serial #.	Mode and duplicate tests.	Automated + reviewed
IMPORT-003	Templates and instructions are generated from the pinned profile version.	Standard and twentieth-field template tests.	Automated + reviewed
IMPORT-004	CSV parsing supports UTF-8 BOM, quotes, commas, Windows endings, and multiple rows.	Parser fixture suite.	Automated + reviewed
IMPORT-005	Mapping accepts case-insensitive labels, field keys, and configured aliases but never guesses ambiguity.	Mapping fixture suite.	Automated + reviewed
IMPORT-006	Duplicate headers, duplicate mappings, and unmapped required fields block validation.	Mapping error tests.	Automated + reviewed
IMPORT-007	Validation classes are Valid, Warning, Blocking Error, and Configuration Error.	Issue contract and UI tests.	Automated + reviewed
IMPORT-008	Required blanks block; optional blanks, including Location, do not block.	All 19 field blank-state tests.	Automated + reviewed
IMPORT-009	Unknown required Owner or Vendor blocks; unknown optional Location warns and imports without location.	Relationship resolution tests.	Automated + reviewed
IMPORT-010	Optional free-text Notes is never validated as a controlled value.	Notes import tests.	Automated + reviewed
IMPORT-011	Controlled values resolve by exact value, normalized value, and active alias.	Lookup matching tests.	Automated + reviewed
IMPORT-012	Every issue links to row and field and offers correction, approved value, authorized add, source correction, or profile repair as applicable.	Guided remediation browser tests.	Automated + reviewed
IMPORT-013	Users may edit, exclude, restore, bulk-correct, and export validation issues without re-uploading.	Session operation tests.	Automated + reviewed
IMPORT-014	Included rows commit atomically and idempotently; any database or conflict error rolls back the batch.	Rollback and retry tests.	Automated + reviewed
IMPORT-015	Successful commit refreshes Search, Inventory, Reports, Activity, import history, and category totals.	Cross-page cache invalidation test.	Automated + reviewed
RPT-001	Report calculations execute server-side and are registry-driven.	API and SQL query tests.	Automated + reviewed
RPT-002	KPI, drilldown, and CSV export share one filtered query contract and reconcile exactly.	Automated reconciliation suite.	Automated + reviewed
RPT-003	Reports include totals, lifecycle availability, category, status, location, owner, vendor, model, and project breakdowns.	Report catalogue tests.	Automated + reviewed
RPT-004	Reports include missing metadata, rework, e-waste, aging, and date-received trends.	Report catalogue tests.	Automated + reviewed
RPT-005	Every KPI and chart element opens its matching paginated asset result.	Browser drilldown tests.	Automated + reviewed
RPT-006	Report responses always initialize KPI, dimensions, trends, rows, and pagination.	Contract and malformed-data tests.	Automated + reviewed
RPT-007	Saved reports store owner, filters, columns, sorting, and compatible profile version.	Save/open/version tests.	Automated + reviewed
RPT-008	CSV export is generated by the server with permission and Activity records.	Download, permission, and audit tests.	Automated + reviewed
RPT-009	Charts provide text/table equivalents and do not use color alone.	Accessibility review.	Automated + reviewed
RPT-010	No report may label incomplete or stale data as current without a visible qualification.	Staleness and configuration tests.	Automated + reviewed
ACT-001	Activity is immutable and records actor, time, action, source, reason, record type, and record ID.	Database trigger and API tests.	Automated + reviewed
ACT-002	Activity records parent import/bulk operation and NVBug/reference context where applicable.	Import and bulk event tests.	Automated + reviewed

Requirement	Binding rule	Required acceptance evidence	Gate
ACT-003	Before/after changes are structured rows, not embedded in narrative reason text.	Mutation contract tests.	Automated + reviewed
ACT-004	Every activity response initializes changes as an array.	Contract and legacy-record tests.	Automated + reviewed
ACT-005	Every event links directly to the affected asset, profile, import, or admin record.	Navigation tests.	Automated + reviewed
ACT-006	Global and asset activity are paginated and filterable.	Pagination and filter tests.	Automated + reviewed
ACT-007	Create, update, status, assignment, transfer, import, profile, lookup, image, export, and print actions are covered.	Action coverage matrix.	Automated + reviewed
ACT-008	Malformed or legacy activity data cannot blank the Activity route.	Defensive rendering tests.	Automated + reviewed
API-001	All public application endpoints are versioned under /api/v1.	Route catalogue test.	Automated + reviewed
API-002	Collections return arrays even when empty; optional aggregate objects are initialized.	Contract schema tests.	Automated + reviewed
API-003	Errors use a stable envelope with status, code, message, details, requestId, and latest conflict state when relevant.	Failure contract tests.	Automated + reviewed
API-004	Pagination uses documented page, limit, total, and deterministic ordering.	Pagination contract tests.	Automated + reviewed
API-005	Asset responses always include nvbugs, mrsOrders, and capacityRequests arrays.	Asset contract tests.	Automated + reviewed
API-006	Import create, fetch, validate, correct, and commit return one documented session envelope.	Import contract tests.	Automated + reviewed
API-007	Version endpoint exposes web, API, schema, import-contract, and package compatibility.	Version contract test through IIS.	Automated + reviewed
API-008	Request IDs appear in logs and user-visible failure states.	End-to-end error correlation test.	Automated + reviewed
API-009	The frontend normalizes responses before rendering and rejects incompatible contracts visibly.	Adapter and compatibility tests.	Automated + reviewed
API-010	API catalogue and frontend requests reconcile with no undocumented call.	Static request-to-route reconciliation.	Automated + reviewed
UX-001	The component system defines typography, spacing, fields, buttons, cards, tabs, sheets, dialogs, tables, and feedback.	Token and component inventory.	Automated + reviewed
UX-002	Pages use professional domain wording and do not expose implementation terminology to users.	Product copy review.	Automated + reviewed
UX-003	Every page has loading, empty, permission, configuration, and failure states.	Page-state browser matrix.	Automated + reviewed
UX-004	Route-level error boundaries prevent one malformed record from blanking a route and offer Retry and Return to Search.	Injected-error browser tests.	Automated + reviewed
UX-005	Overlays have one internal scroll surface, fixed actions, backdrop/Escape/close dismissal, and focus restoration.	Keyboard and viewport tests.	Automated + reviewed
UX-006	Browser Back closes overlays or returns to the prior page without losing committed state.	History navigation tests.	Automated + reviewed
UX-007	Search text, category shortcuts, and applied filters remain independent.	Interaction tests.	Automated + reviewed
UX-008	The NVIDIA logo is official, clickable, and returns to Search while clearing transient state.	Header navigation test.	Automated + reviewed
UX-009	Motion is restrained, communicates state, and respects prefers-reduced-motion.	Motion and reduced-motion tests.	Automated + reviewed
UX-010	Supported widths have no overlap, clipped actions, nested scrollbars, or horizontal page overflow.	Screenshot and overflow suite at five required viewports.	Automated + reviewed
UX-011	Forms expose definitions, required state, constraints, and direct recovery guidance without clutter.	Usability and accessibility review.	Automated + reviewed
UX-012	Labels use Code 128 SVG, isolate print content, create one label-sized page per asset, and record print activity.	Barcode scan and print tests.	Automated + reviewed
OPS-001	Production runtime is IIS, one Fastify Windows service, and PostgreSQL; Node runtime is bundled.	Service and process inventory.	Automated + reviewed
OPS-002	Start order is PostgreSQL, Fastify, then IIS; stop order is IIS, Fastify, then PostgreSQL when database maintenance requires it.	Runbook and maintenance test.	Automated + reviewed

Requirement	Binding rule	Required acceptance evidence	Gate
OPS-003	Operations provide install, migrate, start, stop, restart, status, maintenance, logs, cleanup, and health commands.	Script execution matrix.	Automated + reviewed
OPS-004	Normal changes use component deltas; full baseline is only for clean installation or disaster recovery.	Package manifest review.	Automated + reviewed
OPS-005	Delta updater verifies version and hashes before stopping services and publishes only declared components.	Tamper and component-scope tests.	Automated + reviewed
OPS-006	Temporary packages and temporary previous components are removed after success; no persistent backup or snapshot is created by this framework.	Filesystem cleanup test.	Automated + reviewed
OPS-007	Maintenance mode protects users during component switch and never masks a failed recovery.	Maintenance and failure tests.	Automated + reviewed
OPS-008	Logs are structured, rotate, contain request IDs and record links, and separate service, update, audit, and health concerns.	Log format and navigation tests.	Automated + reviewed
OPS-009	Health includes liveness, readiness, version compatibility, database reachability, and dependency state.	Health endpoint and degraded-state tests.	Automated + reviewed
OPS-010	External monitoring alerts on unavailability, sustained latency, error rate, and database connection failure.	Monitor evidence and alert drill.	Automated + reviewed
OPS-011	Support documentation defines ownership, severity, diagnosis, escalation, and recovery.	Runbook tabletop exercise.	Automated + reviewed
OPS-012	A release ledger records version, package hash, operator, components, migrations, acceptance evidence, and outcome.	Ledger and package reconciliation.	Automated + reviewed
TEST-001	Requirement IDs appear in automated test names and the generated conformance report.	Test report parse.	Automated + reviewed
TEST-002	HTTP 200 alone cannot approve a release; browser rendering and business workflows are mandatory.	Acceptance report includes browser and workflow gates.	Automated + reviewed
TEST-003	Every route is tested directly and through navigation under all three roles.	Role-by-route browser matrix.	Automated + reviewed
TEST-004	Database tests cover keys, constraints, indexes, rollback, restart persistence, ownership, and deletion behavior.	Database acceptance report.	Automated + reviewed
TEST-005	Five concurrent sessions cannot silently overwrite asset changes.	409 conflict concurrency report.	Automated + reviewed
TEST-006	Import tests cover all 19 fields and one administrator-added twentieth field.	Import conformance report.	Automated + reviewed
TEST-007	Report KPI, drilldown, and export reconcile for every standard dimension.	Report reconciliation report.	Automated + reviewed
TEST-008	Visual acceptance covers 1920x1080, 1366x768, 1280x720, 768x1024, and 390x844.	Screenshot evidence set.	Automated + reviewed
TEST-009	Performance targets are p95 <500 ms search/detail, <800 ms mutation, and <2 s reports at expected volume.	Load report with five concurrent sessions.	Automated + reviewed
TEST-010	A candidate with any failed applicable requirement is not packaged or assigned a release version.	Conformance and release ledger gate.	Automated + reviewed

13.1 Mandatory scenario walkthroughs

Scenario	Required result
Field creation	Add optional twentieth field, activate profile, verify all eight enabled surfaces, then deprecate.
Lookup addition	Reject unknown controlled value, authorize Add As New Value with reason, revalidate matching rows and audit.
Import correction	Map CSV, repair source/staged value, bulk-correct repeated issue, export validation report, resume session.
Duplicate serial	Create and import attempts block; Update Existing resolves exactly one asset.
Concurrent edit	Five sessions edit one revision; first succeeds, later stale mutation returns structured 409.
Report export	KPI count equals drilldown total and exported row count for every dimension.
Production failure	Stop API, degrade database, publish bad web component in test; users see recoverable states and alerts correlate logs.
Restart	Restart API, PostgreSQL, and server; data persists; IIS returns compatible web/API and no prototype processes.

13.2 Paperwork quality gate

- Exactly 19 standard fields, eight required, eleven optional, and six lifecycle statuses reconcile with the registry and database.

- The ERD/table catalogue reconciles with every PostgreSQL table, primary key, foreign key, index, constraint, owner, grant, and delete rule.
- Every frontend request reconciles with one documented API route and response contract.
- Every requirement has an owner, acceptance evidence, and pass/fail result. No undefined term or contradictory rule remains.
- Every PDF page is rendered to an image and visually inspected for readable tables, diagrams, headings, page numbers, links, and approval fields.

13.3 Integrated go-live decision

Gate	Pass condition
Database	invmgmt catalog, keys, grants, migrations, constraints, indexes, rollback, restart persistence all pass.
Identity and permissions	All role/page/endpoint combinations pass and actor/role spoof attempts fail.
Asset workflows	Create, update, Assign, Return, Transfer, Change Status, Archive, Restore, relationships, images, and conflicts pass.
Import	All 19 fields plus one added field, mappings, corrections, resume, rollback, idempotency, and refresh pass.
Reporting	Every KPI, drilldown, chart table, and export reconciles exactly.
Activity	Every mutation is immutable, structured, paginated, and directly navigable.
UI	All roles, routes, states, overlays, printing, and five viewports pass with zero console exceptions.
Operations	Reboot, maintenance, service recovery, logs, monitoring, delta update, cleanup, and version compatibility pass.
Performance	p95 targets pass with at least five concurrent sessions and representative Day-0 volume.
Decision	All applicable requirements are Passed; Product Owner records go-live approval in the release ledger.

14. Current Candidate Assessment

No production approval

The Saturday build remains the implementation candidate. This chapter records repository evidence and known gaps; it does not certify production behavior.

14.1 Evidence present

- React/TypeScript compiled web, Fastify API, PostgreSQL baseline, IIS scripts, Windows service wrapper, component delta tooling, and version contract are present.
- The baseline defines 19 fields, six lifecycle statuses, relational assets/models, hierarchical locations, roles/permissions, persistent import structures, reports, and immutable Activity structures.
- API types initialize reference arrays, activity changes, report structures, import session states, and conflict/error metadata.
- A route error boundary and focused unit tests exist, and backend checks have previously completed successfully in the development environment.

14.2 Release blockers and conformance backlog seeds

Area	Current evidence	Requirement	Required correction
Release provenance	inventory-production is currently untracked in the enclosing Git repository.	GOV-008	Block release until committed, reviewed, and tagged.
Schema ownership	Candidate baseline creates application objects in public; framework target is invmgt.	DATA-001, DATA-015	Controlled data-preserving schema migration and grant reconciliation.
Acceptance depth	Operations acceptance currently emphasizes HTTP and version checks.	TEST-002, TEST-003	Add browser and business workflow gates through IIS.
Frontend coverage	Current tests do not render every complete route or role state.	TEST-003, TEST-008	Page-by-page Playwright matrix and visual evidence.
Import contract history	Observed production failures show prior frontend/backend import session drift.	API-006, IMPORT-001..015	Single envelope contract plus server/browser conformance tests.
Report trust	Current evidence does not prove KPI, drilldown, and export reconciliation.	RPT-002	Shared-query reconciliation test for every standard dimension.
Activity robustness	Observed blank-page failures demonstrate legacy/malformed activity risk.	ACT-004, ACT-008	Response normalization and malformed-record route tests.
Dependency security	A complete dependency audit result is not present in current evidence.	OPS-009, TEST-010	Produce package inventory and high/critical vulnerability gate.
Production monitoring	No external availability/latency/error-rate monitor is evidenced.	OPS-010	Install monitor and complete alert drill before broad go-live.

14.3 Approval consequence

Approval does not approve the Saturday build for go-live. Approval freezes the rules used to compare the candidate. The next artifact is one conformance backlog ordered by database contract, API contract, registry propagation, permissions, imports, reports, activity, UI stability, and operations. Existing PostgreSQL data and application files are preserved while those gaps are addressed through controlled changes.

15. Five-Year Roadmap

Horizon	Theme	Controlled outcomes
0-6 months	Stabilize	Approve framework; baseline Git/tag; invmgmt migration; API normalization; full route tests; import/report/activity conformance; operational monitoring.
6-12 months	Identity and support	Introduce Microsoft Entra ID SSO through a governed amendment; retain DB roles; improve support dashboard, alerts, and service-level reporting.
Year 2	Scale and analytics	Capacity tests, query tuning, materialized reporting where justified, saved views, stock-risk forecasting, and approved data retention.
Year 3	Integration	Governed NVBug/MRS/capacity integrations, directory synchronization, notification channels, and API consumer contracts.
Year 4	Resilience	Recovery exercises, high availability decision, database lifecycle upgrades, archival policy, and security maturity review.
Year 5	Renewal	Architecture reassessment, framework v2 decision, dependency modernization, usability research, and formal five-year reapproval.

15.1 Roadmap governance

Roadmap items are not pre-approved implementation. Each item requires a change request, impact analysis, acceptance criteria, security and support review, and Product Owner approval. The roadmap may be reordered by amendment without weakening current binding requirements.

15.2 Backup plan if the candidate cannot conform

The framework remains stable even if the Saturday candidate fails. The fallback is not another technology change: preserve the approved PostgreSQL contract and framework, isolate the nonconforming component, and replace only that component behind the same API/data contracts. If a component cannot meet its acceptance gate after two controlled correction cycles, the Product Owner commissions a replacement implementation for that component using the same requirements, fixtures, and conformance tests. Data migration or destructive reset requires a separate approved decision.

16. Architectural Reference Evidence

The following sources were accessed during framework preparation. Their principles inform the requirements; Inventory Project remains a custom implementation.

Reference	URL	Applied principle
Snipe-IT Custom Fields	https://snipe-it.readme.io/docs/custom-fields	Fieldsets group custom fields, attach to models, control required state/order, and drive forms/views.
Snipe-IT Importing Assets	https://snipe-it.readme.io/docs/importing-assets	Explicit column mapping and preconfigured fields prevent accidental bad-data structures.
InvenTree Data Import	https://docs.inventree.org/en/latest/concepts/data_import/	Persistent server import sessions, separate create/update modes, related-record resolution, and resumability.
InvenTree Permissions	https://docs.inventree.org/en/stable/settings/permissions/	Group/role permissions and server-side enforcement.
InvenTree Stock	https://docs.inventree.org/en/stable/stock/	Stock item as an instance, hierarchical locations, status, and tracking history.
InvenTree Data Export	https://docs.inventree.org/en/stable/concepts/data_export/	Exports derive from the same governed data and filtering surfaces.
InvenTree Stock Views	https://docs.inventree.org/en/latest/app/stock/	Transfer, barcode, labels, detail views, and stock operations.

16.1 Internal candidate evidence

Repository evidence reviewed includes package scripts, React routes and API adapter, Fastify route modules and types, PostgreSQL baseline and import migrations, registry/lifecycle/reference/report modules, automated tests, IIS/service/update scripts, release manifests, and existing operations/import/support documents under inventory-production.

17. Approval

Approval effect

Signature approves this framework as the governing baseline. It does not by itself approve the current application for go-live. Implementation resumes only through the conformance backlog and acceptance process defined here.

Product Owner	Gaurav Mehta
Decision	☐ Approved ☐ Rejected ☐ Approved with attached change record
Signature	
Approval date	
Effective version	Inventory Project Product and Engineering Framework v1.0
Validity	Five years from approval date unless superseded by an approved amendment
Comments / change record	

Document integrity after approval

Record the approved PDF SHA-256, repository commit/tag, and conformance backlog link in the controlled document register. Any content change produces a new revision and requires the amendment procedure.